

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Báo cáo Thực tập chuyên ngành

Đánh giá phôi đại VA mức video từ nội soi mũi họng với phân
đoạn hai vùng

Giảng viên hướng dẫn:

Giảng viên đồng hướng dẫn:

Lớp:

Sinh viên:

Mã sinh viên:

Dr. Lê Đức Trọng

Dr. Đỗ Thành Công

QH-2032-I/CQ-I-CS3

Phạm Mai Anh

23021469

Hà Nội, 2026

MỤC LỤC

Lời cảm ơn	3
1 Giới thiệu chung	4
1.1 Giới thiệu công việc	4
1.2 Giới thiệu qua bài toán	5
2 Yêu cầu bài toán	5
2.1 Bài toán lớn: đánh giá phi đại VA ở mức video	5
2.2 Phần bài toán sinh viên giải quyết	6
2.3 Tiêu chí đánh giá	6
3 Tóm tắt lý thuyết, giải pháp, thuật toán	7
3.1 Các lý thuyết, giải pháp, thuật toán liên quan	7
3.2 Cách giải quyết của sinh viên	8
3.2.1 Bộ nhớ không gian Kalman một khe	8
3.2.2 Huấn luyện bộ cập nhật	9
3.2.3 Hạ tầng dữ liệu và đánh giá	10
3.3 Liên hệ và so sánh với các cách đã có	11
4 Mô tả phần mềm cài đặt	11
4.1 Cấu trúc mã nguồn	11
4.2 Cài đặt	12
4.3 Quy trình chạy	12
5 Kết quả đạt được, hướng phát triển	12
5.1 Kết quả đạt được	12
5.1.1 Thiết lập thực nghiệm	12
5.1.2 So sánh đã cố định trên PolypGen	14
5.1.3 Đóng góp đo được của Kalman memory và cổng hiện diện	14
5.1.4 Diễn giải và giới hạn	15

5.1.5	Đánh giá hai vùng trên REFUGE	16
5.2	Kỹ năng và kiến thức thu thập được	17
5.3	Hướng phát triển tiếp theo	17
Tài liệu tham khảo		18

Lời cảm ơn

EmTÔI xin chân thành cảm ơn các giảng viên của Trường Đại học Công nghệ, đặc biệt là thầy Lê Đức Trọng và thầy Đỗ Thành Công, đã hướng dẫn và góp ý cho quá trình thực hiện đề tài. Em cũng cảm ơn các bạn và những người đã hỗ trợ trao đổi chuyên môn trong thời gian thực tập.

1 Giới thiệu chung

1.1 Giới thiệu công việc

Phì đại VA (adenoid hypertrophy) là một nguyên nhân thường gặp gây tắc nghẽn đường thở trên ở trẻ em. Trong thực hành, bác sĩ đánh giá mức độ phì đại bằng nội soi mũi họng: ống soi đi qua hốc mũi, và mức độ VA che lấp lòng mũi họng được ước lượng bằng mắt. Cách đánh giá này phụ thuộc người đọc và thường chỉ dựa trên một vài khung hình được chọn. Đề tài thực tập hướng tới việc tự động hóa phép đo này ở mức *video*: phân đoạn vùng VA và vùng đường thở mũi họng trên từng khung hình, tính tỷ lệ tắc nghẽn trên các khung hợp lệ, rồi tổng hợp thành một đánh giá cho cả video.

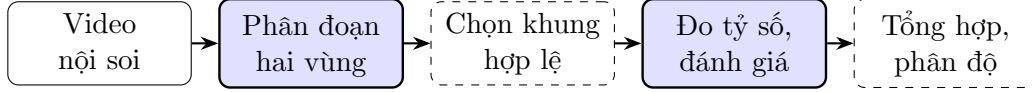
Trong đợt thực tập, đóng góp kỹ thuật trung tâm của tôi là thiết kế và cài đặt **bộ nhớ lặp có nhận biết độ tin cậy** thay cho ngăn hàng FIFO bảo khung của MedSAM2 [1]. Bộ nhớ này giữ một trạng thái không gian cố định, dùng cập nhật Kalman có nhiễu học được để quyết định tin trạng thái cũ hay quan sát mới, và dùng **cổng hiện diện** để không ghi/lập lại đối tượng khi nó đã biến mất. YOLOv8 cung cấp quan sát độc lập để phát hiện lại khi tracker lệch. Xung quanh đóng góp đó, công việc gồm (i) xây dựng chuỗi huấn luyện, suy luận và đánh giá; (ii) đánh giá chất lượng lan truyền theo thời gian và đánh giá hai vùng có sai số tỷ lệ; (iii) thiết kế giao thức thực nghiệm, chạy thí nghiệm, phân tích lỗi và ghi nhận trung thực kết quả đến thời điểm hiện tại.

Ý tưởng hướng đến một điểm yếu cơ bản của phân đoạn video y tế: một mặt nạ sai nếu được ghi vô điều kiện vào FIFO có thể tự khuếch đại ở các frame sau. Kalman filter cung cấp cơ chế định lượng bất định để làm mượt việc ghi bộ nhớ; cổng hiện diện biến “đối tượng có/không” thành quyết định có ảnh hưởng trực tiếp đến xuất mặt nạ, object pointer và gain ghi nhớ. Vì thế, tiềm năng của phương pháp không chỉ là thay một cấu trúc dữ liệu, mà là tách ba quyết định thường bị gộp trong bank gốc: *có đối tượng không, quan sát này đáng tin đến đâu, và ghi bao nhiêu vào trạng thái dài hạn*.

Ở thời điểm báo cáo, dự án chưa có bộ dữ liệu video VA đã gán nhãn, cũng chưa có giao thức lâm sàng xác định công thức tắc nghẽn và ngưỡng phân độ. Vì vậy mọi thực nghiệm dùng dữ liệu thay thế: PolypGen [4] và CholecSeg8k [5] cho bài toán lan truyền phân đoạn theo thời gian, REFUGE [6] cho đường xử lý hai vùng và sai số tỷ số. Báo cáo không tuyên bố kết quả lâm sàng trên VA.

1.2 Giới thiệu qua bài toán

Chuỗi xử lý mục tiêu của dự án AdeSEG được minh họa ở Hình 1:



Hình 1: Chuỗi xử lý mục tiêu. Ô tô đậm: phần sinh viên đã cài đặt và đánh giá; ô nét đứt: phần cần giao thức lâm sàng và dữ liệu VA để hoàn thiện.

Bài toán tự động hoá khoanh vùng trước khi tính tỷ lệ amidan và vòm họng khó ở hai điểm. Thứ nhất, liên quan đến các tính chất của video nội soi, phân đoạn phải *ổn định theo thời gian*: chuyển động nhanh của camera dẫn đến hiện tượng bị mờ, nhoè (blurring); một số vùng bị quá sáng hoặc quá tối do ánh sáng từ ống soi và một số chi tiết khác có thể gây nhiễu cho mô hình, làm mô hình khoanh vùng sai; sự thay đổi về hình dạng và kích thước của amidan qua các khung hình trong 1 video: nhỏ, lớn khác nhau trong mỗi khung hình, bị che khuất 1 phần, không xuất hiện trong 1 vài khung hình rồi lại xuất hiện lại (object reappearing). Một mô hình phân đoạn video có bộ nhớ như MedSAM2 có thể “bám nhầm” đối tượng và tích lũy lỗi qua các khung, gây ảnh hưởng nghiêm trọng đến khả năng của mô hình nếu như không có các biện pháp chọn lọc phù hợp. Đề tài giải quyết bằng việc đưa ra một phương pháp lưu trữ và cập nhật các thông tin có chọn lọc thay thế cho bộ nhớ FIFO của MedSAM2.

2 Yêu cầu bài toán

2.1 Bài toán lớn: đánh giá phì đại VA ở mức video

Cho một video nội soi gồm các khung $I_1, \dots, I_T$, hệ thống cần dự đoán cho mỗi khung hai mặt nạ nhị phân: A_t cho VA và B_t cho đường thở mũi họng không bị che. Một khung chỉ được dùng để đo khi hai mặt nạ cùng kích thước, có diện tích đủ lớn và tỷ số hữu hạn. Tỷ lệ được tính theo công thức:

$$r_t = \frac{|A_t|}{|A_t| + |B_t|}$$

Cùng ý nghĩa với tỷ số A/N mà Cai và cộng sự [3] dùng để phân độ: nhỏ ($< 50\%$), trung bình ($50-75\%$), lớn ($> 75\%$). Từ các khung hợp lệ, tỷ số của video được tổng hợp bằng trung vị hoặc trung bình, rồi ánh xạ sang mức độ phì đại bằng một dãy ngưỡng tăng chặt. Công thức tỷ số và ngưỡng là lựa chọn của giao thức lâm sàng, cần có sự đồng thuận của bác sĩ.

Đầu vào. Video đã sắp theo thứ tự khung. Mỗi khung là một dòng manifest JSONL với các trường `split`, `video_id`, `frame_index`, `image`, `mask`. Mặt nạ là ảnh chỉ số: 0 nền, 1 đường thờ, 2 VA (khớp với mã hóa mức xám 0/128/255 của Cai và cộng sự sau khi chuyển đổi). Việc chia tập phải theo video hoặc bệnh nhân, không theo khung, để tránh rò rỉ dữ liệu.

Đầu ra. Ở mức khung: mặt nạ từng vùng, diện tích, tỷ số, kiểu tỷ số và cờ `valid_frame`. Ở mức video: tỷ số tổng hợp và, khi có ngưỡng, nhãn phân độ.

2.2 Phần bài toán sinh viên giải quyết

Trong bài toán lớn ở trên, tôi giải quyết hai phần cụ thể:

(a) Bộ nhớ theo thời gian cho phân đoạn video. MedSAM2 lan truyền mặt nạ từ khung có prompt sang các khung sau nhờ một ngăn hàng bộ nhớ chứa tối đa bảy khung gần nhất (FIFO). Khi một khung bị phân đoạn sai, bộ nhớ sai đó được ghi vào ngăn hàng và ảnh hưởng tới các khung sau; khi đối tượng biến mất, mô hình vẫn có xu hướng dự đoán đối tượng. Yêu cầu đặt ra là thay FIFO bằng một kiến trúc bộ nhớ có kích thước cố định với hai ý tưởng chính: (1) **cập nhật Kalman** cân bằng trạng thái cũ và quan sát mới theo bất định học được; (2) **cổng hiện diện/độ tin cậy** ngăn mặt nạ và object pointer không đáng tin đi vào chuỗi hồi tiếp. Mục tiêu thực nghiệm là giảm dương tính giả và mất theo dõi mà không đánh đổi quá nhiều Dice. Ràng buộc: giữ nguyên trọng số MedSAM2 (frozen), chỉ thay cơ chế bộ nhớ, để so sánh công bằng.

(b) Đánh giá hai vùng và sai số đo. Xây dựng đường xử lý hai nhãn từ mặt nạ mã hóa mức xám đến tỷ số, và một bộ đánh giá báo cáo đồng thời Dice/IoU từng vùng và tỷ lệ. Mục đích là kiểm tra trực tiếp liệu Dice tốt có thể đi kèm sai số tỷ lệ lớn hay không, trước khi áp dụng cho tập dữ liệu amidan thực tế sắp tới.

2.3 Tiêu chí đánh giá

Phân đoạn được đo bằng Dice và IoU theo từng chuỗi rồi lấy trung bình, cùng Dice riêng trên các khung có đối tượng. Hiện diện được đo bằng tỷ lệ dương tính giả trên khung rỗng (FP khung rỗng). Tích lũy lỗi được đo bằng tỷ lệ khung có đối tượng bị “mất” ($\text{Dice} < 0.1$), độ dài trung bình một đoạn mất, tỷ lệ đoạn được khôi phục và tỷ lệ mất tới cuối video. Độ bền được

đo bằng Dice ở 1–10 khung sau khi đối tượng tái xuất hiện, ở khung chuyển động cao và theo mức chiếu sáng. Đo hai vùng dùng MAE, RMSE và tương quan Pearson/Spearman giữa tỷ số dự đoán và tỷ số thật, cùng độ chính xác phân độ và hệ số kappa khi có ngưỡng. So sánh cặp theo chuỗi dùng kiểm định Wilcoxon, và tập kiểm tra chỉ được chạy một lần cho mỗi phương pháp đã đóng băng.

3 Tóm tắt lý thuyết, giải pháp, thuật toán

3.1 Các lý thuyết, giải pháp, thuật toán liên quan

SAM2 và MedSAM2. SAM2 [2] phân đoạn video bằng một bộ mã hóa ảnh (Image Encoder), một khối *memory attention* cho đặc trưng khung ảnh hiện tại chú ý tới bộ nhớ các khung hình đã lưu, một bộ giải mã mặt nạ (Mask Decoder) và một bộ mã hóa bộ nhớ (Memory Encoder). Sau mỗi khung, bộ mã hóa bộ nhớ nén **đặc trưng ảnh và mặt nạ dự đoán** thành một bản đồ đặc trưng $64 \times 32 \times 32$ (cùng kích thước với kết quả đặc trưng của phần Image Encoder); bản đồ này được đẩy vào ngân hàng bộ nhớ gồm khung prompt (Bounding Box, Điểm, Mask) của frame đầu tiên, cộng tối đa sáu khung gần nhất, cùng các *object pointer*. Bộ giải mã (Decoder) còn xuất một *object score* cho biết đối tượng có mặt hay không. MedSAM2 [1] tinh chỉnh SAM2 trên dữ liệu ảnh và video y tế, giữ nguyên kiến trúc bộ nhớ này. Điểm yếu đã biết: nếu không có chiến lược chọn lọc frame cẩn thận, những frame bị khoanh vùng sai hoặc không rõ ràng sẽ gây ảnh hưởng đến chất lượng của mô hình trong việc khoanh vùng các frame sau (error accumulation).

Cải tiến bộ nhớ của SAM2. Hầu hết các bài báo trước đều nhận ra điểm yếu của phần *Memory Bank* mà thiếu đi sự chọn lọc frame tốt/xấu. SAM2Long [7] giữ nhiều nhánh giả thuyết bộ nhớ và chọn nhánh tốt theo điểm tích lũy. DAM4SAM [8] tách bộ nhớ thành phần gần đây và phần chống nhiễu (distractor-aware), chỉ ghi khung khi đủ tin cậy. EMA-SAM [9] dùng trung bình trượt có trọng số theo độ tin cậy cho bộ nhớ của SAM2 trong ảnh y tế. SAMURAI [10] dùng bộ lọc Kalman, nhưng trên *chuyển động hộp bao* để chọn mặt nạ ứng viên và lọc bộ nhớ, không phải trên chính đặc trưng bộ nhớ. Tuy nhiên, không có sự so sánh giữa các phương pháp chọn lọc khung ảnh trong video, độc lập.

Bộ nhớ lặp kích thước cố định. Trong phân đoạn đối tượng video, RDE-VOS [11] thay ngân hàng bộ nhớ ngày càng lớn bằng một embedding động được cập nhật lặp; LiVOS [12] dùng ma trận trạng thái tuyến tính có

cổng (gated linear). Cả hai bài báo đều chứng minh 1 trạng thái động được cập nhật qua các frame hoàn toàn có thể mang lại kết quả tương đương hoặc hơn so với bộ nhớ mặc định FIFO gồm stack các thông tin của các khung hình trước, trong khi vẫn tối ưu được về mặt tính toán.

Bộ lọc Kalman - Kalman Filtering. Bộ lọc Kalman [13] ước lượng trạng thái ẩn từ các quan sát nhiễu bằng hai bước dự đoán và cập nhật; gain K cân bằng giữa trạng thái cũ và quan sát mới theo phương sai của chúng. Recurrent Kalman Networks [14] đưa cập nhật Kalman với hiệp phương sai đường chéo vào không gian đặc trưng học được; KalmanNet [15] học trực tiếp gain bằng mạng nơ-ron; Recursive KalmanNet [16] học gain và một thừa số Cholesky của hiệp phương sai bằng mạng GRU, với cập nhật hiệp phương sai dạng Joseph. Khi quan sát có thể không đến từ đối tượng (đối tượng vắng mặt, hoặc bị nhầm với vật khác), bộ lọc liên kết dữ liệu xác suất (probabilistic data association, PDA) chỉ áp dụng cập nhật theo xác suất β rằng quan sát thật sự là của đối tượng.

Bộ phát hiện đối tượng. YOLOv8 là bộ phát hiện một giai đoạn cho ra hộp bao và độ tin cậy trên từng ảnh, không có trạng thái theo thời gian; vì vậy lỗi của nó độc lập với lỗi tích lũy của bộ nhớ MedSAM2.

3.2 Cách giải quyết của sinh viên

3.2.1 Bộ nhớ không gian Kalman một khe

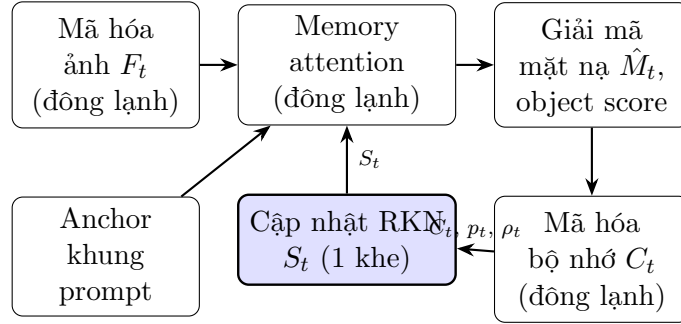
Thay vì ngân hàng FIFO bảy khung của MedSAM2, hệ thống đọc bộ nhớ của khung prompt (anchor) và một trạng thái lặp $S_t \in \mathbb{R}^{64 \times 32 \times 32}$. Quan sát C_t là bộ nhớ do MedSAM2 mã hóa từ đặc trưng ảnh và mặt nạ của khung t . Bộ cập nhật dùng KalmanFilter của Recursive KalmanNet với $F = H = 1$ trên từng phần tử của S_t . Cập nhật được liên kết dữ liệu xác suất:

$$S_t = S_t^- + \beta_t K_t (C_t - S_t^-), \quad \beta_t = p_t \rho_t,$$

trong đó p_t là xác suất hiện diện đã hiệu chỉnh và ρ_t là độ tin cậy của mặt nạ theo dõi. Đây là điểm then chốt của ý tưởng: khi khung vắng mặt hoặc quan sát không đáng tin, β_t nhỏ nên trạng thái không bị kéo về một quan sát sai mà chủ yếu chỉ dự đoán; khi quan sát phù hợp, gain Kalman cho phép ghi có trọng số thay vì ghi toàn phần như FIFO. Bốn mạng GRU học hiệu chỉnh hiện diện, ρ_t , nhiễu quá trình Q từ thay đổi đặc trưng ảnh, và nhiễu quan sát R từ xác suất mặt nạ, hộp detector và khoảng cách tới anchor. Cổng hiện diện đồng thời hiệu chỉnh object score và làm yếu object pointer không tin cậy. MedSAM2 vẫn đông lạnh.

Detector YOLOv8 cung cấp một quan sát độc lập: hộp có độ tin cậy ít nhất 0.5 nhưng không khớp mặt nạ (IoU hộp dưới 0.5) sẽ kích hoạt tái phát hiện. Khi đó mặt nạ được giải mã trực tiếp từ hộp detector trên đặc trưng không dùng bộ nhớ, rồi được xuất và ghi vào trạng thái. Object pointer cũng được chặn theo độ tin cậy. Các ngưỡng tái phát hiện được kiểm tra trên các video huấn luyện trước khi chạy C6. Hình 2 minh họa phân bộ nhớ lặp; khối tô đậm là phần mới.

Đối chứng RDE-VOS. Để kiểm tra liệu lợi ích (nếu có) đến từ cấu trúc bộ nhớ lặp nói chung hay từ RKN, hệ thống còn tích hợp RDE-VOS. Lớp `RDEMemoryUpdate` dùng trực tiếp khối `SAM` và phép nén `compress_key` từ `external/RDE-VOS-CVPR2022`; trạng thái được cập nhật theo $RDE_t = \text{compress_key}(\text{SAM}([RDE_{t-1}, C_t]))$. Ở chế độ `two-frames-compress`, nó đọc prompt, RDE và khung nhớ gần nhất, và chỉ nén lại mỗi ba khung. RDE-VOS trong bảng kết quả là đối chứng bộ nhớ lặp chính thức, chưa huấn luyện (seed 0), không phải một bản cài đặt lại EMA.



Hình 2: Luồng dữ liệu của bộ nhớ một khe: MedSAM2 đọc anchor của khung prompt cộng một trạng thái lặp thay vì ngân hàng FIFO bảy khung; chỉ khối tô đậm là mới, toàn bộ MedSAM2 giữ đông lạnh.

3.2.2 Huấn luyện bộ cập nhật

Bộ cập nhật được huấn luyện trên các clip tự nhiên 24 khung của seq1–seq12, trong đó một nửa clip có đoạn vắng mặt; seq13–seq15 được giữ lại để chọn checkpoint. Hàm mất mát gồm mất mát mặt nạ ở các khung sau prompt, BCE hiện diện có trọng số theo mục tiêu $J = \text{Dice} - 0.5 \text{ FP}$, BCE mềm cho ρ_t theo IoU của mặt nạ theo dõi, và Gaussian likelihood cho bộ nhớ mang theo. Sau 500 bước, checkpoint “full” giảm FP trên seq13–seq15 từ 0.878 (không huấn luyện) xuống 0.022, đổi lại Dice từ 0.797 xuống 0.778.

Vì cùng tập được dùng để chọn checkpoint và báo cáo, tập phát triển này chỉ dùng để lựa chọn, không phải kiểm định cuối cùng.

3.2.3 Hạ tầng dữ liệu và đánh giá

Manifest và thứ tự khung. Tên tệp khung của PolypGen có hậu tố số với độ dài khác nhau, nên sắp theo bảng chữ cái là sai (khung 104 đứng trước khung 69). Phiên bản đầu của manifest mắc lỗi này và đưa 18/23 chuỗi vào sai thứ tự thời gian. Tôi phát hiện lỗi, sửa `build_polypgen_manifest.py` sắp theo số, loại bỏ mọi kết quả từ phiên bản cũ, thêm kiểm thử khóa thứ tự đúng, và xác nhận pipeline mới tái lập chính xác các lần chạy lưu trữ (Dice gộp: phát triển native 0.481/Kalman 0.514; C6 native 0.621/Kalman 0.609).

Lỗi đồng bộ và kiểm chứng tương đương. Trên Apple MPS, việc MedSAM2 offload trạng thái với `non_blocking` đọc dữ liệu trước khi sao chép xong và làm hỏng các kết quả đầu; tôi sửa bằng cách giữ trạng thái trên thiết bị khi không dùng CUDA. Mã `native_pointers.py` được kiểm chứng trùng khớp từng bit với object pointer của MedSAM2 gốc, nên khác biệt giữa các phương pháp chỉ đến từ bộ nhớ không gian.

Tái lập kết quả. Các JSON theo phương pháp trong thư mục kết quả so sánh tái lập đúng các trị số báo cáo: native trên C6 là Dice/FP/mất = 0.599/0.486/0.283, trùng hàng C6 đã lưu; trên seq13–seq15, hai checkpoint đã huấn luyện cũng khớp số xác thực lúc huấn luyện. Suy luận trực tiếp và script so sánh cho cùng Dice trên seq15. Các CSV theo khung lưu cờ tái phát hiện và ρ , cho phép kiểm tra đường suy luận và phân tích lỗi.

Mặt nạ hai vùng. Mặt nạ ba lớp mã hóa mức xám (0/128/255) nếu đọc như mặt nạ nhị phân sẽ gộp đường thở và VA thành một lớp. `build_mask_manifest.py` chuyển chúng sang PNG chỉ số bằng ánh xạ giá trị gần nhất (để nhiều nén JPEG không tạo lớp mới), ví dụ 0:0 128:1 255:2 cho dữ liệu kiểu Cai và cộng sự. `evaluation/two_region.py` tính Dice/IoU từng vùng và tính tỷ số dự đoán và tỷ số thật bằng *cùng* một `ratio_mode`, nên không thể vô tình so sánh hai tỷ số định nghĩa khác nhau. Cờ `-nested_labels` của `infer.py` prompt nhân k như hợp của các nhân $\geq k$ (đĩa thị = vành + cup; mũi họng = VA + đường thở) rồi lấy vùng ngoài trừ vùng trong.

3.3 Liên hệ và so sánh với các cách đã có

Bảng 1 đặt giải pháp cạnh các hướng liên quan. Cập nhật Kalman được đặt trực tiếp lên bản đồ bộ nhớ không gian của MedSAM2; các mạng GRU học hiện diện, độ tin cậy và nhiễu, còn MedSAM2 giữ đông lạnh. Thí nghiệm cũng có RDE-VOS làm đối chứng bộ nhớ lặp và một baseline detector không có bộ nhớ.

Bảng 1: So sánh định tính với các hướng liên quan.

Phương pháp	Cơ chế bộ nhớ	Kích thước	Quan hệ với đề tài
SAM2	/	FIFO: prompt + 6	Baseline chính
MedSAM2	khung gần nhất, ghi mọi khung	7 khung	
SAM2Long	Nhiều nhánh giả thuyết bộ nhớ	Nhiều bank	Chọn bộ nhớ, không gộp
DAM4SAM	Bộ nhớ gần đây + bộ nhớ chống nhiễu	Bank	Ghi có điều kiện
SAMURAI	Kalman trên hộp bao, chọn mặt nạ	Bank	Kalman trên chuyển động, không trên bộ nhớ
EMA-SAM	Trung bình trượt theo độ tin cậy	Cố định	Hướng liên quan, chưa so sánh trực tiếp
RDE-VOS, LiVOS	Trạng thái lặp / tuyến tính có cổng	Cố định	Cùng ý tưởng kích thước cố định
RKN, KalmanNet	Cập nhật Kalman trong không gian học được	–	Nguồn phương trình cập nhật
Đề tài	Anchor + 1 trạng thái, RKN có liên kết dữ liệu và detector	Cố định	–

Kết quả ở mục 5 cho thấy mô hình đầy đủ vượt MedSAM2 gốc theo mục tiêu J trên C6, nhưng baseline detector không bộ nhớ cũng tốt hơn mô hình đầy đủ. Do đó bằng chứng hiện có quy cải thiện cho detector (tái phát hiện và hiệu chỉnh hiện diện), không cho phần bộ nhớ Kalman. RDE-VOS và RKN vẫn có ưu điểm kiến trúc là trạng thái kích thước cố định, nhưng đây không phải bằng chứng về cải thiện phân đoạn.

4 Mô tả phần mềm cài đặt

4.1 Cấu trúc mã nguồn

Mã nguồn viết bằng Python/PyTorch và giữ MedSAM2 đông lạnh. Hai thành phần ngoài được tích hợp có chủ đích: `external/RecursiveKalmanNet`

cung cấp `KalmanFilter` và `GRUNetwork` cho RKN; `external/RDE-VOS-CVPR2022` cung cấp SAM và phép nén khóa cho baseline RDE-VOS. Các lớp trong `modeling/` nối các thành phần này với bộ nhớ không gian của MedSAM2, huấn luyện và suy luận; chúng không chạy các chương trình huấn luyện gốc của hai dự án ngoài. Bảng 2 tóm tắt các mô-đun liên quan.

4.2 Cài đặt

Phần mềm chạy trên CUDA, Apple MPS hoặc CPU. Cài đặt gồm: tạo môi trường ảo Python và chạy `pip install -r requirements.txt`; đặt checkpoint `MedSAM2_latest.pt` vào `checkpoints/`; đặt dữ liệu dưới `data/` và dựng manifest bằng các script `build*_manifest.py`; kiểm tra bằng `python3 -m pytest tests`.

4.3 Quy trình chạy

Suy luận đọc manifest, chọn tập qua `-split`, nhãn qua `-label_ids`; không truyền `-memory_checkpoint` thì dùng bank native của MedSAM2:

```
python3 scripts/infer.py \
  --sam2_cfg configs/sam2.1_hiera_t512.yaml \
  --sam2_checkpoint checkpoints/MedSAM2_latest.pt \
  --manifest data/polypgen_sequence.jsonl \
  --split test --label_ids 1 \
  --memory_checkpoint outputs/memory_update.pt \
  --detector checkpoints/polypgen_yolov8n.pt \
  --output_dir runs/rkn
```

Với bộ nhớ native hoặc RDE-VOS, cờ `-redetect` bật tái phát hiện từ detector. Với dữ liệu hai vùng, mỗi nhãn được lan truyền trong một lượt riêng rồi gộp theo logit lớn nhất, hoặc theo thứ tự lồng nhau với `-nested_labels`. Mỗi lần suy luận ghi mặt nạ, bản ghi prompt, bộ nhớ giữ lại, tốc độ khung hình và CSV chẩn đoán theo khung (trong đó có cờ tái phát hiện và độ tin cậy ρ).

5 Kết quả đạt được, hướng phát triển

5.1 Kết quả đạt được

5.1.1 Thiết lập thực nghiệm

PolypGen gồm 23 chuỗi nội soi đại tràng có mặt nạ polyp (2,012 khung lan truyền: 1,689 có polyp, 323 rỗng), chú thích thừa và có nhiều đoạn polyp

Bảng 2: Các mô-đun chính.

Mô-đun	Chức năng
modeling/kalman_memory.py	Bộ cập nhật RKN/Kalman với hiện diện, độ tin cậy và nhiễu học được
modeling/recurrent_memory.py	Thay bank MedSAM2 bằng anchor và trạng thái lặp; tái phát hiện
modeling/rde_memory.py	Đối chứng RDE-VOS với trạng thái lặp
modeling/detector.py	Quan sát hộp và độ tin cậy từ YOLOv8
external/RecursiveKalmanNet/Algo/	KalmanFilter, GRUNetwork được RKN tái sử dụng
external/RDE-VOS-CVPR2022/model/	SAM và compress_key được RDE-VOS tái sử dụng
modeling/native_pointers.py	Object pointer, trùng khớp MedSAM2 gốc
training/memory_trainer.py	Huấn luyện và hàm mất mát cho RKN/RDE-VOS
datasets/video.py	Đọc manifest, giải mã mặt nạ
scripts/build*_manifest.py	Manifest PolypGen, sắp khung theo số; mặt nạ mức xám sang PNG chỉ số
scripts/infer.py,	Suy luận native/RKN/RDE-VOS;
scripts/train_memory.py	huấn luyện bộ cập nhật
scripts/evaluate_polypgen.py	Chạy cùng một giao thức so sánh cho mọi baseline và ghi JSON theo phương pháp
scripts/summarize_polypgen_results.py	Tổng hợp J và kiểm định Wilcoxon cấp từ JSON đã lưu
evaluation/temporal.py	Chỉ số theo khung, hiện diện, mất theo dõi
evaluation/two_region.py	Dice/IoU từng vùng, sai số tỷ số, kappa
scripts/analyze_runs.py	Dựng lại mọi bảng PolypGen từ CSV
adenoid/	Đo tỷ số, chọn khung hợp lệ, phân độ
tests/test_data_pipeline.py	Kiểm thử thứ tự khung, giải mã mặt nạ

ra khỏi khung hình. MedSAM2 giữ đông lạnh; prompt là hộp bao chặt của mặt nạ thật ở khung có polyp đầu tiên, nên kết quả phản ánh chất lượng *lan truyền* sau prompt, không phải hiệu năng phát hiện tự động. Giao thức phát triển hiện được mã hóa trong `protocols/polypgen_recurrent_v1.json`: seq1–seq12 để huấn luyện, seq13–seq15 để chọn thiết kế và seq16–seq23 (C6) chỉ được gắn nhãn *exploratory*. C6 *không* phải tập hoàn toàn chưa thấy: nó đã nằm trong giao thức 23 chuỗi trước khi giao thức (a) được định nghĩa, và sau đó được dùng cho nhiều so sánh. Một tập video ngoài hoàn toàn mới vẫn cần thiết cho kiểm định khóa; mọi số liệu PolypGen dưới đây hiện chỉ dùng một seed.

5.1.2 So sánh đã cố định trên PolypGen

Câu hỏi trung tâm của phần này là liệu bộ nhớ Kalman có cổng hiện diện có thể biến một chuỗi dự đoán dễ tích lũy lỗi thành một quá trình cập nhật có chọn lọc hay không. Đóng góp cài đặt không chỉ là thay một công thức cập nhật: mô-đun **recurrent memory** thay bank bẫy khung của MedSAM2 bằng anchor và một trạng thái không gian cố định. Trong RKN, KalmanFilter/GRUNetwork từ `external/RecursiveKalmanNet` học Q , R , hiện diện p và độ tin cậy ρ ; pp quyết định mức ghi quan sát vào trạng thái. Nhờ vậy, cổng hiện diện là một phần của cơ chế bộ nhớ, không chỉ là hậu xử lý mặt nạ. Detector cung cấp quan sát độc lập để hiệu chỉnh hiện diện và tái phát hiện khi tracker không khớp. **RDE-VOS** dùng phép nén hồi quy từ `external/RDE-VOS-CVPR2022` làm đối chứng recurrent-memory. Vì MedSAM2 đông lạnh, mọi khác biệt dưới đây đến từ đường bộ nhớ lặp, các đầu RKN/RDE và detector, không phải do fine-tune backbone.

Các phương pháp và ngưỡng tái phát hiện trong bảng chính được cố định trước khi chạy so sánh C6. Tất cả dùng cùng prompt: hộp bao chặt của mặt nạ thật ở khung có polyp đầu tiên, vì vậy đây là đánh giá *lan truyền* sau prompt. Tập phát triển là seq13–seq15; C6 là seq16–seq23 (8 chuỗi, 372 khung *lan truyền*, 56 khung *rỗng*). Số liệu là trung bình theo chuỗi. “Mất” là tỷ lệ khung polyp có $\text{Dice} < 0.1$, và $J = \text{Dice}_{\text{polyp}} - 0.5 \text{FP}_{\text{rỗng}}$. `evaluate_polypgen.py` ghi JSON theo khung, `run.json` ghi vai trò giao thức và `summary.json` ghi tổng hợp/kiểm định; do C6 đã được xem trong quá trình phát triển, không kết quả nào trên C6 được gọi là kiểm định khóa.

5.1.3 Đóng góp đo được của Kalman memory và cổng hiện diện

Bảng 5 làm rõ đóng góp theo cơ chế. So với MedSAM2 native, RKN đầy đủ tăng J 0.308 trên cả 8 chuỗi và giảm FP 0.398. So với RKN không huấn luyện, các đầu RKN học được giảm FP 0.435 mà gần như không đổi Dice

Bảng 3: So sánh đã cố định: Dice khung polyp / FP khung rộng / mất / J (trung bình theo chuỗi).

Phương pháp	seq13–15				C6 seq16–23			
	Dice	FP	Mất	J	Dice	FP	Mất	J
Native MedSAM2	0.393	0.681	0.558	0.053	0.599	0.486	0.283	0.355
Native + tái phát hiện	0.810	0.773	0.099	0.423	0.775	0.562	0.104	0.494
Detector, không bộ nhớ	0.815	0.023	0.117	0.804	0.742	0.086	0.193	0.699
RDE-VOS	0.408	0.955	0.523	-0.069	0.575	0.544	0.316	0.303
RDE-VOS + tái phát hiện	0.805	0.920	0.093	0.345	0.763	0.587	0.118	0.469
RKN không huấn luyện	0.794	0.804	0.113	0.392	0.709	0.523	0.191	0.447
Chỉ hiện diện (RKN)	0.433	0.011	0.511	0.428	0.544	0.038	0.386	0.525
RKN đầy đủ	0.778	0.022	0.129	0.767	0.707	0.088	0.206	0.663

Bảng 4: So sánh cặp RKN đầy đủ trên C6 ($n = 8$, Wilcoxon; hiệu RKN trừ baseline).

So sánh	Dice khung polyp	FP khung rộng	J
RKN đầy đủ – native	+0.109 (7/1), $p = 0.078$	-0.398 (0/6), $p = 0.031$	+0.308 (8/0), $p = 0.008$
RKN đầy đủ – native + tái phát hiện	-0.068 (2/6), $p = 0.15$	-0.473 (0/6), $p = 0.031$	+0.169 (6/2), $p = 0.15$
RKN đầy đủ – RDE-VOS + tái phát hiện	-0.055 (2/6), $p = 0.039$	-0.499 (0/5), $p = 0.062$	+0.194 (5/3), $p = 0.20$
RKN đầy đủ – detector không bộ nhớ	-0.035 (1/7), $p = 0.20$	+0.002 (1/1), $p = 1.0$	-0.036 (1/7), $p = 0.20$

(-0.001); điều này phù hợp với trực giác rằng các đầu p , ρ , Q và R chủ yếu từ chối quan sát/mặt nạ sai. Ablation “chỉ hiện diện” giữ đầu hiện diện học được nhưng bỏ cập nhật bộ nhớ học được, chặn pointer và tái phát hiện; vì thế nó cho thấy cổng hiện diện rất mạnh để triệt FP, nhưng không cô lập gain Kalman.

Đối chứng trực tiếp hơn giữ *nguyên* checkpoint RKN đầy đủ, detector, ngưỡng tái phát hiện, cổng hiện diện và cổng pointer; chỉ thay cập nhật Q/R học được bằng $S_t = S_t^- + \alpha\beta_t(C_t - S_t^-)$ với gain cố định. Quét $\alpha \in \{0.10, 0.25, 0.50, 0.75, 0.90\}$ trên seq13–seq15 chọn $\alpha = 0.25$ theo J (0.773). Khi chạy *thăm dò* trên C6, RKN học được đạt Dice/FP/ $J = 0.707/0.088/0.663$, so với control gain cố định 0.695/0.097/0.647; chênh $\Delta J = +0.017$ (5/3 chuỗi, Wilcoxon $p = 0.945$). Vì đối chứng này không có ý nghĩa và C6 đã được dùng để phát triển, dữ liệu hiện tại **không chứng minh** lợi ích độc lập của gain Kalman học được. Nó chỉ cho thấy cơ chế có thể được kiểm tra công bằng mà không làm thay detector hay cổng hiện diện.

5.1.4 Diễn giải và giới hạn

Trên C6, RKN đầy đủ tốt hơn MedSAM2 native theo J : +0.308 ở cả 8 chuỗi ($p = 0.008$); FP giảm 0.398 ($p = 0.031$), còn Dice tăng 0.109 nhưng chưa đạt ý nghĩa ở $n = 8$ ($p = 0.078$). Đây là bằng chứng rằng **hệ thống**

Bảng 5: Ablation trên C6: tác động của hệ thống RKN (hiệu RKN đầy đủ trừ baseline; trung bình theo chuỗi).

Baseline	Δ Dice khung polyp	Δ FP khung rỗng	ΔJ	Diễn giải
Native MedSAM2	+0.109 ($p = 0.078$)	-0.398 ($p = 0.031$)	+0.308 ($p = 0.008$)	Tác động tổng thể so với bank gốc
RKN không huấn luyện	-0.001 ($p = 0.95$)	-0.435 ($p = 0.062$)	+0.216 ($p = 0.055$)	Huấn luyện chủ yếu giảm FP
Chỉ hiện diện (RKN)	+0.163 ($p = 0.016$)	+0.050 ($p = 0.50$)	+0.138 ($p = 0.11$)	Hệ thống đầy đủ so với ablation
RDE-VOS + tái phát hiện	-0.055 ($p = 0.039$)	-0.499 ($p = 0.062$)	+0.194 ($p = 0.20$)	RKN so với recurrent-memory đối chứng
Detector không bộ nhớ	-0.035 ($p = 0.20$)	+0.002 ($p = 1.0$)	-0.036 ($p = 0.20$)	Không chứng minh lợi ích độc lập của bộ nhớ

gồm trạng thái lặp, detector, tái phát hiện và cổng hiện diện/độ tin cậy giảm dương tính giả so với bank native trong cấu hình này. Về mặt ý tưởng, Kalman filter vẫn cung cấp cách mở rộng tự nhiên sang video VA: bất định cao hoặc vắng mặt sẽ hãm ghi bộ nhớ, còn quan sát tin cậy sẽ hiệu chỉnh trạng thái thay vì thay thế nó. Native MedSAM2 tái lập chính xác kết quả C6 đã lưu: Dice/FP/mất = 0.599/0.486/0.283.

Giới hạn của diễn giải này rất quan trọng: đây chưa phải bằng chứng rằng *chỉ riêng* gain Kalman tạo cải thiện phân đoạn. Baseline detector không bộ nhớ tốt hơn RKN đầy đủ trên cả hai tập: tại C6, Dice 0.742 so với 0.707 và J 0.699 so với 0.663, tốt hơn ở 7/8 chuỗi (không có ý nghĩa thống kê). Fixed-gain control cũng chỉ cho $\Delta J = +0.017$, $p = 0.945$. RKN hơn native+tái phát hiện và RDE-VOS+tái phát hiện về J chủ yếu nhờ ít FP hơn, trong khi Dice của hai baseline đó cao hơn RKN lần lượt 0.068 và 0.055. RDE-VOS trong bảng là đối chứng chưa huấn luyện; comparison RDE đã huấn luyện, cùng seed và cùng tập phát triển, chưa hoàn tất ở thời điểm viết báo cáo. Suy luận phù hợp là detector, tái phát hiện, cổng hiện diện và bộ nhớ Kalman *cùng* tạo hệ thống tốt hơn MedSAM2 native; đóng góp độc lập của Kalman cần nhiều seed, RDE đã huấn luyện và một tập video ngoài chưa từng được xem.

Detector và bộ nhớ cũng thất bại ở các video khác nhau: trên seq18 Dice của RKN là 0.58, trong khi detector không bộ nhớ là 0.19; trên seq23 các giá trị là 0.49 và 0.87. Một cơ chế kết hợp có thể hữu ích, nhưng phải được thiết kế và chọn hoàn toàn trên tập phát triển rồi mới đánh giá trên dữ liệu VA. C6 đã được dùng cho so sánh này và không nên dùng tiếp để chọn thiết kế.

5.1.5 Đánh giá hai vùng trên REFUGE

Dữ liệu VA dự kiến theo Cai và cộng sự [3] dùng mặt nạ ba lớp mã hóa 0/128/255. REFUGE (ảnh đáy mắt) dùng cùng mã hóa với hai vùng kề nhau không chồng lấn (vành đĩa thị và cup), nên tỷ số cup/(cup + vành) có cùng dạng với tỷ số A/N. Mặt nạ được chuyển bằng ánh xạ 255:0 128:1 0:2; MedSAM2 gốc được prompt bằng hộp bao từ nhãn thật, theo hai thiết kế.

Thiết kế thứ nhất có lỗi: hộp bao nhãn thật của vành chính là hộp của

Bảng 6: Hai vùng trên REFUGE val (400 ảnh).

Prompt	Dice cup	Dice vành	MAE tỷ số	RMSE tỷ số	Pearson	Spearman
Mỗi nhãn một lượt (hộp vành = hộp đĩa)	0.734	0.886	0.109	0.138	0.74	0.86
-nested_labels (đĩa và cup; vành = đĩa - cup)	0.917	0.924	0.019	0.024	0.98	0.98

cả đĩa thị, và khi mỗi nhãn chạy một lượt rồi “logit lớn nhất thắng”, dự đoán vành nuốt mất cup. Tỷ lệ cup dự đoán thấp hơn nhãn thật trên *cả 400* ảnh (sai số có dấu trung bình -0.109), và sai số tỷ số đi theo Dice cup (Spearman $\rho = -0.74$). Đó là lỗi do cách prompt, không phải sự lệch giữa Dice và phép đo. Thiết kế lồng nhau (prompt đĩa và cup, lấy vành = đĩa - cup) khớp với cấu trúc A/N, nơi mỗi hòng = VA + đường thở; với nó, sai số tỷ số chỉ 0.019 và ngay cả các ảnh có cả hai Dice ≥ 0.9 (249 ảnh) cũng chỉ có MAE 0.015. **Như vậy REFUGE không cho thấy sự lệch giữa Dice và phép đo**, và chưa có cơ sở cho một hàm mất mát nhận biết phép đo.

5.2 Kỹ năng và kiến thức thu thập được

Về kiến thức: kiến trúc bộ nhớ của SAM2/MedSAM2, lý thuyết bộ lọc Kalman, liên kết dữ liệu xác suất và các hướng cải tiến bộ nhớ trong phân đoạn video. Về lập trình: mở rộng một mã nguồn nghiên cứu lớn bằng mixin, xử lý dữ liệu video y tế, tìm và sửa lỗi thứ tự khung và đồng bộ MPS, tích hợp detector như một quan sát độc lập, và viết kiểm thử cho đường dữ liệu. Về phương pháp: cố định phương pháp trước khi chạy C6, dùng baseline detector không bộ nhớ để kiểm tra quy kết nguyên nhân, tái lập số xác thực và kết quả native từ dữ liệu đã lưu, và ghi nhận trung thực kết quả âm.

5.3 Hướng phát triển tiếp theo

1. **Dữ liệu VA dạng video** cùng giao thức tỷ số và ngưỡng phân độ; đây sẽ là tập kiểm tra sạch đầu tiên.
2. **Kết hợp detector và bộ nhớ**: học hoặc quy tắc chọn nguồn dự đoán khi detector đáng tin cậy hoặc khi detector thất bại; mọi lựa chọn phải làm trên tập phát triển.
3. **Đánh giá trên dữ liệu VA**: giữ cố định phương pháp sau khi chọn trên tập phát triển và dùng video VA như kiểm tra sạch.
4. **Hàm mất mát nhận biết phép đo** chỉ khi dữ liệu VA cho thấy Dice và sai số tỷ số lệch nhau.

Tài liệu tham khảo

- [1] MedSAM2: Segment Anything in 3D Medical Images and Videos. <https://arxiv.org/abs/2504.03600>.
- [2] SAM 2: Segment Anything in Images and Videos. <https://arxiv.org/abs/2408.00714>.
- [3] Cai và cộng sự (2024). Deep Learning-Based Quantification of Adenoid Hypertrophy and Its Correlation with Apnea-Hypopnea Index in Pediatric Obstructive Sleep Apnea. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11687100/>.
- [4] PolypGen: A multi-center polyp detection and segmentation dataset. <https://arxiv.org/abs/2106.04463>.
- [5] CholecSeg8k: A Semantic Segmentation Dataset for Laparoscopic Cholecystectomy. <https://arxiv.org/abs/2012.12453>.
- [6] REFUGE: Retinal Fundus Glaucoma Challenge, phân đoạn đĩa thị và cup trên ảnh đáy mắt.
- [7] SAM2Long: Enhancing SAM 2 for Long Video Segmentation. ICCV 2025.
- [8] DAM4SAM: A Distractor-Aware Memory for Visual Object Tracking with SAM2. <https://arxiv.org/abs/2509.13864>.
- [9] EMA-SAM. <https://arxiv.org/abs/2510.18213>.
- [10] SAMURAI: Adapting Segment Anything Model for Zero-Shot Visual Tracking with Motion-Aware Memory. <https://arxiv.org/abs/2411.11922>.
- [11] Recurrent Dynamic Embedding for Video Object Segmentation (RDE-VOS). <https://arxiv.org/abs/2205.03761>.

- [12] LiVOS: Light Video Object Segmentation with Gated Linear Matching. <https://arxiv.org/abs/2411.02818>.
- [13] R. E. Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering.
- [14] Recurrent Kalman Networks: Factorized Inference in High-Dimensional Deep Feature Spaces. <https://arxiv.org/abs/1905.07357>.
- [15] KalmanNet: Neural Network Aided Kalman Filtering for Partially Known Dynamics. <https://arxiv.org/abs/2107.10043>.
- [16] Recursive KalmanNet: Deep Learning-Augmented Kalman Filtering for State Estimation with Consistent Uncertainty Quantification. EUSIPCO 2025. <https://arxiv.org/abs/2506.11639>.